

第1章 绪论

算法分析的基本原则

1. 正确性

- 定义：在给定有效输入后，算法经过有限时间的计算并产生正确的答案，就称算法是正确的。

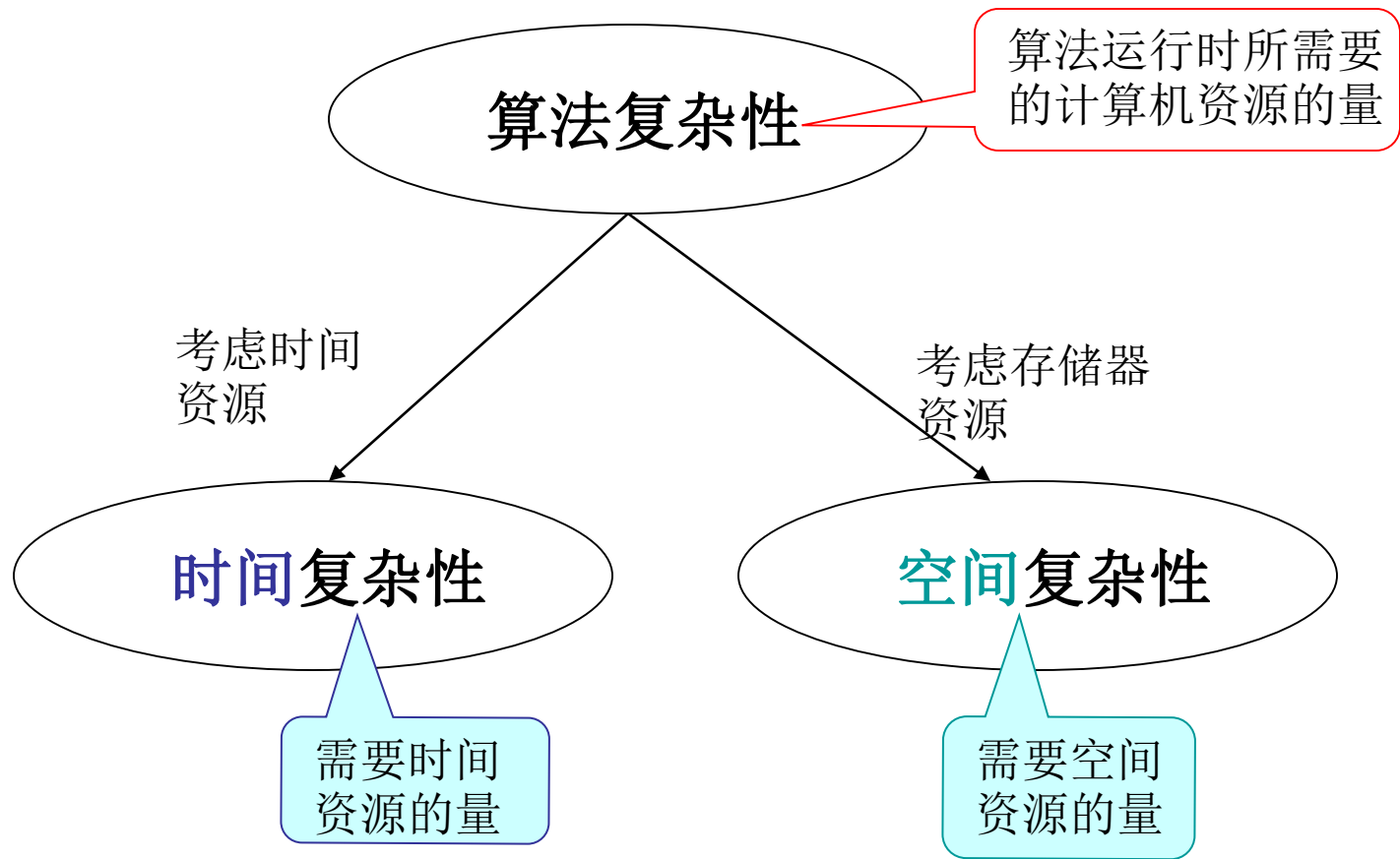
2. 工作量——时间复杂性分析

- 计量工作量的标准：对于给定问题，该算法所执行的基本运算的次数。
- 基本运算的选择：根据问题选择适当的基本运算。

3. 占用空间——空间复杂性分析

- 两种占用：
 - 存储程序和输入数据的空间
 - 存储中间结果或操作单元所占用空间——额外空间

算法复杂性



时间复杂性函数的具体化？

- 一个算法的时间复杂度（**time complexity**）是指算法运行所需的时间。
- 对于给定的N、I和A，如何导出 **$T=T(N, I)$** 和 **$S=S(N, I)$** ？
- **$T(N, I)$** 表示算法在一台抽象的计算机上运行需要的时间。设此抽象计算机的元运算有**k**种，分别记为 **$O_1, O_2, \dots, O_k$** 。每执行一次这些运算所需要的时间分别为 **$t_1, t_2, \dots, t_k$** 。设算法用到元运算 **O_i** 的次数为 **e_i** （是**N**和**I**的函数），则

$$T(N, I) = \sum_{i=1}^k t_i e_i(N, I)$$

- 时间复杂度

- 一般情况下，不可能也没有必要总是对规模为 n ，基本运算 O_i 的执行次数 e_i 分别进行统计分析。
- $T(N, I)$ 还需进一步简化，只在某些有代表性的合法输入中去统计相应的 e_i 来评价其复杂性。
- 一般只考虑三种情况下的时间性：最坏情况、最好情况和平均情况下的复杂性，分别记为 $T_{\max}(N)$ 、 $T_{\min}(N)$ 和 $T_{\text{avg}}(N)$

四种渐近意义下的符号

- 四种渐近意义下的符号

- O

- Ω

- θ

- o

假设 $f(N)$ 和 $g(N)$ 是定义在正整数集上的正函数

符号“O”的运算规则

$$(1) O(f) + O(g) = O(\max(f, g));$$

$$\text{例: } O(n^2) + O(n) = O(\max(n^2, n)) = O(n^2)$$

$$(2) O(f) + O(g) = O(f + g);$$

$$(3) O(f)O(g) = O(fg);$$

$$\text{例: } O(n^2) \times O(n) = O((n^2 \times n)) = O(n^3)$$

$$(4) \text{如果 } g(N) = O(f(N)), \text{ 则 } O(f) + O(g) = O(f);$$

$$\text{例: } 2n + 1 = O(n), \text{ 则 } O(n) + O(2n + 1) = O(2n + 1 + n) = O(n)$$

$$(5) O(Cf(N)) = O(f(N)), \text{ } C \text{ 为常数};$$

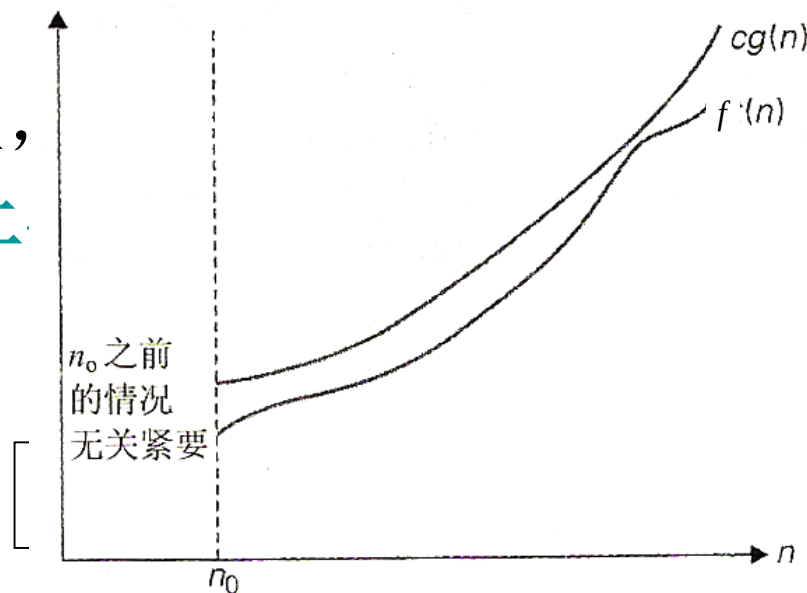
$$(6) f = O(f);$$

四种渐近意义下的符号之“O”

如果存在正的常数 C 和自然数 N_0 ，使得当 $N \geq N_0$ 时， $f(N) \leq Cg(N)$ ，则称函数 $f(N)$ 当 N 充分大时有上界，且 $g(N)$ 是它的一个上界，记为 $f(N) = O(g(N))$ 。这时还说 $f(N)$ 的阶不高于 $g(N)$ 的阶。

注：用**O**评估算法的复杂性，充分大时的**一个上界**。该**上精确**，结果就越有价值。

g通常为下列简单函数单项形式：



四种渐近意义下的符号之“ Ω ”

如果存在正的常数 C 和自然数 N_0 ，使得当 $N \geq N_0$ 时， $f(N) \geq Cg(N)$ ，则称函数 $f(N)$ 当 N 充分大时有下界，且 $g(N)$ 是它的一个下界，记为 $f(N) = \Omega(g(N))$ 。这时还说 $f(N)$ 的阶不低于 $g(N)$ 的阶。

注：用 Ω 评估算法的复杂性，所得到的只是当规模充分大时的一个下界。该下界的阶越高，评估就越精确，结果就越有价值。

四种渐近意义下的符号之“ θ ”

$$f(N) = \theta(g(N))$$

当且仅当 $f(N) = O(g(N))$ 且 $f(N) = \Omega(g(N))$

称 $f(N)$ 和 $g(N)$ 同阶

习题 1-1 函数的渐近表达式

求下列函数的渐近表达式：

$3n^2 + 10n$; $n^2/10 + 2^n$; $21 + 1/n$; $\log n^3$; $10\log 3^n$ 。

分析与解答：

$$3n^2 + 10n = O(n^2);$$

$$n^2/10 + 2^n = O(2^n);$$

$$21 + 1/n = O(1);$$

$$\log n^3 = O(\log n);$$

$$10\log 3^n = O(n)。$$

按渐近阶排列表达式

按照渐近阶从低到高的顺序排列以下表达式： $4n^2$ ， $\log n$ ， 3^n ， $20n$ ， 2 ， $n^{2/3}$ 。又 $n!$ 应该排在哪一位？

分析与解答：

按渐近阶从低到高，函数排列顺序如下： 2 ， $\log n$ ， $n^{2/3}$ ， $20n$ ， $4n^2$ ， 3^n ， $n!$ 。

课后练习： 1-4

假设某算法的 $T(n)=3 \times 2^n$ ，在某台机上实现并完成该算法的时间为 t 秒。

如果有一台机的运行速度是前一台机的64倍，则在这台机上用同一算法在 t 秒内能解多大规模的问题？

$$t = 3 \times 2^n = 3 \times 2^{n_1} \div 64$$

$$\rightarrow n_1 = n + 6$$

课后练习： 1-6

对于下列各组函数 $f(n)$ 和 $g(n)$ ，确定 $f(n) = O(g(n))$ 或 $f(n) = \Omega(g(n))$ 或 $f(n) = \theta(g(n))$ ，并简述理由。

(1) $f(n) = \log n^2$; $g(n) = \log n + 5$

(2) $f(n) = \log n^2$; $g(n) = \sqrt{n}$

(3) $f(n) = n$; $g(n) = \log^2 n$

(4) $f(n) = n \log n + n$; $g(n) = \log n$

(5) $f(n) = 10$; $g(n) = \log 10$

(6) $f(n) = \log^2 n$; $g(n) = \log n$

(7) $f(n) = 2^n$; $g(n) = 100n^2$

(8) $f(n) = 2^n$; $g(n) = 3^n$

第2章 递归与分治策略

分治法的基本思想

- 分治法的基本思想

- 将一个规模为 n 的问题分解为 k 个规模较小的子问题，这些子问题互相独立并且与原问题相同。通过递归地求解这些子问题，然后再将各个子问题的解合并，就可以实现对原问题的求解。

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

通过解递归方程

$$\Rightarrow T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j f(n/m^j)$$

主方法

根据这两项对结果的影响不同，分成三种情况讨论

设 $a \geq 1, b > 1$ 为常数， $f(n)$ 为函数

$$T(n) = aT(n/b) + f(n),$$

$T(n)$ 为非负整数

1. $f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0,$

那么 $T(n) = \Theta(n^{\log_b a})$

2. $f(n) = \Theta(n^{\log_b a}),$

那么 $T(n) = \Theta(n^{\log_b a} \log n)$

3. $f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0,$

且对于某个常数 $c < 1$ 和所有的充分大的 n 有

$$af(n/b) \leq cf(n),$$

那么 $T(n) = \Theta(f(n))$

前一项比后一项的阶高，对结果的影响大，忽略后一项

两项同阶

后一项比前一项的阶高，对结果的影响大，忽略前一项

学习要点:

理解递归的概念。

掌握设计有效算法的分治策略。

通过下面的范例学习分治策略设计技巧。

- ⇒ (1) 二分搜索技术;
- (2) 大整数乘法;
- (3) Strassen矩阵乘法;
- (4) 棋盘覆盖;
- ⇒ (5) 合并排序和快速排序;
- ⇒ (6) 线性时间选择;
- (7) 最接近点对问题;
- (8) 循环赛日程表。

第三章:动态规划

动态规划算法的基本思想

- 动态规划算法的基本思想

- 其基本思想与分治算法的思想类似——分而治之

- 与分治法的不同之处

- 分解后的子问题往往不互相独立；

- 采用记录表的方法来保存所有已解决问题的答案

- 对每个子问题都只计算一次，不管该子问题以后是否会被用到，都将其保存到一张表中，避免每次遇到各个子问题都要重复计算答案。

- 一个最优化多步决策问题适合用动态规划法求解有两个要素：最优子结构特性和重叠子问题。

动态规划算法的一般步骤

- 动态规划算法的一般步骤

1. 找出最优解的性质，并刻画其结构特征；
2. 递归地定义最优值；
3. 以自底向上的方式计算出最优值；
4. 根据计算最优值时得到的信息，构造最优解；

动态规划算法
的基本步骤

如只需求出最优值，则步骤4可省略；

如要求解问题的最优解，则必须执行步骤4

动态规划算法求解思路

- 动态规划算法求解思路

- 将待求解的问题分解成若干个相互联系的子问题，先求解子问题，然后从这些子问题的解得到原问题的解；
- 对于重复出现的子问题，只在第一次遇到的时候对它进行求解，并把答案保存起来，让以后再次遇到时直接引用答案，不必重新求解。
- 动态规划算法将问题的解决方案视为一系列决策的结果，**与贪婪算法不同的是：**
 - 在贪婪算法中，每采用一次贪婪准则，便做出一个不可撤回的决策；
 - 在动态规划算法中，**还要考察每个最优决策序列中是否包含一个最优决策子序列**，即问题是否具有最优子结构性质。

矩阵连乘问题----最优解的结构

- 将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 记作 $A[i:j]$
 - 把问题转化成考察 $A[1:n]$ 的最优计算次序问题
 - 设计算次序在 $A[k]$ 处将矩阵断开最优，则总计算量为： $A[1:k]$ 的计算量加上 $A[k+1:n]$ 的计算量，再加上 $A[1:k]$ 和 $A[k+1:n]$ 相乘的计算量。
- 关键特征
 - $A[1:n]$ 的最优计算次序所包含的计算矩阵子链 $A[1:k]$ 和 $A[k+1:n]$ 的次序也是最优的
 - 问题的最优解包含了其子问题的最优解，这种性质称为**最优子结构性**。

最优子结构性是该问题可用动态规划算法求解的第一个标志

第4章 贪心算法

- 理解贪心算法的一般理论
- 通过应用范例学习贪心设计策略。

⇒ (1) 活动安排问题;

(2) 最优装载问题;

⇒ (3) 哈夫曼编码;

⇒ (4) 单源最短路径;

⇒ (5) 最小生成树;

(6) 多机调度问题。

- 贪心算法(greedy algorithm)：作出在当前看来最好的选择
 - 贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择。
 - 希望贪心算法得到的最终结果也是整体最优的。虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路径问题，最小生成树问题等。
 - 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。

贪心算法的基本要素

本节着重讨论可以用贪心算法求解的问题的一般特征。

对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？一般需具有2个重要的性质：**贪心选择性质**和**最优子结构性质**。

贪心选择性质

- 所谓**贪心选择性质**是指所求问题的**整体最优解**可以通过一系列**局部最优**的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。
 - 贪心选择：只考虑对当前问题的最佳选择，不是考虑子问题的结果。
 - 动态规划：每一步都要选择，选择依赖每一个子问题的解；
- 动态规划算法通常以**自底向上**的方式解各子问题，而贪心算法则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。
- 对每个贪心算法，几乎总是会有一个相对更复杂的动态规划解。
- 对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

贪心选择性质的确定

- 贪心选择性质的确定

- 对于一个具体问题，要考察该问题是否具有贪心选择性质，必须证明每一步所做出的贪心选择最终将获得问题的整体最优解。
 - 通常可以用类似于证明活动安排问题的贪心选择时所采用的方法来证明。

首先考察问题的一个整体最优解，并证明可修改这个最优解，使其以贪心选择开始。

在做出贪心选择后，原问题简化为规模更小的类似子问题。

用数学归纳法证明，通过每一步做贪心选择，最终可得到问题的全局最优解。

贪心算法的基本要素

最优子结构性质

- 当一个问题的最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。
- 在活动安排问题中，最优子结构的性质表现在：若A是关于E的活动安排问题包含活动1的一个最优解，则相容活动集合 $A' = A - \{1\}$ 是关于 $E' = \{i \in E: s_i \geq f_1\}$ 的活动安排问题的一个最优解。

贪心算法与动态规划算法的差异

- 贪心算法和动态规划算法都要求问题具有最优子结构性质，这是2类算法的一个共同点。
- 对于具有**最优子结构**的问题应该选用贪心算法还是动态规划算法求解？是否能用动态规划算法求解的问题也能用贪心算法求解？

- 首先将各物品依重量递增序（即也是价值递减序）排列，然后依照价值递减顺序选择物品装入背包，直到背包装不下下一件物品为止。
- 这里贪心算法的贪心选择策略是：每次总是选择价值最大（同时重量也最小）的物品，然后检查是否可以装入背包。

第5章 回溯法

问题的解空间

- 问题的解空间

- 应明确问题的解空间的定义

- 问题的解空间至少应包含问题的一个（最优解）。

- 对问题解空间进行组织

- 通常组织为树或图的形式。

- 有利于回溯法对整个解空间的搜索

回溯法的基本思想

- 回溯法的基本思想

- 在确定解空间的组织结构后，回溯法从开始结点（根结点）出发，以**深度优先方式搜索**整个解空间。这个开始结点成为**活结点**，同时也成为**当前的扩展结点**。
- 在当前扩展结点处，搜索向纵深方向移至一个新结点。这个新结点成为新的活结点，并成为扩展结点。
 - 如果在当前扩展结点处不能再向纵深方向移动，则当前扩展结点就成为**死结点**。此时，应**往回移动（回溯）**到最近的活结点处，并使该结点成为当前的扩展结点。
- 回溯法按上述方式**递归**地在解空间中搜索，**直到找到所要求的解或解空间中以误活结点为止**。

如何避免回溯法的无效搜索

- 如何避免回溯法的无效搜索

- 用约束函数在扩展结点处剪去不满足约束的子树；

- 见0-1背包问题

- 用限界函数剪去得不到最优解的子树；

- 见TSP问题

- 以上两类函数统称为剪枝函数

回溯法的主要步骤

- 回溯法的主要步骤

- 针对所给问题，定义问题的解空间；
- 确定易于搜索的解空间结构；
- 以深度优先方式搜索解空间，并在搜索过程中用剪枝函数避免无效搜索。

实例分析

- ⇒ • 旅行商问题
- 符号三角形问题
- ⇒ • N皇后问题
- 最大团问题
- ⇒ • 图的m着色问题
- 圆排列问题
- 电路板排列问题
- 连续邮资问题

第6章 分支限界法

分支限界法

- 分支限界法

- 与回溯法类似，都是在问题的解空间树上搜索问题解的算法；
- 求解目标：找出满足约束条件的解
 - 可行解或最优解
- 搜索策略
 - 根据限界函数值，剔除那些导致不可行解或非最优解的子结点，使搜索过程仅限制在剩余的分支内进行；

分支限界法 VS 回溯法

- 相同点:

- 两者在进行问题求解前，都需要完成解空间的定义和组织;
- 都是通过解空间搜索来寻找问题的解;

- 回溯法和分支限界法的：

- 共同之处：

- 用于解空间的搜索；
 - 可作为通用问题求解方法；
 - 通过约束剪裁解空间的结点，减小搜索空间；

- 区别之处：

- 采用的数据结构：堆栈**VS**队列（优先队列、堆）
 - 搜索方法：深度优先**VS**广度优先（额外优先函数）
 - 通常的应用：寻找满足约束--所有解**VS**一个解（或特定前提下的最优解）
 - 消耗的存储空间：深度优先算法只需要记录堆栈中的结点；广度优先算法需要保存的结点数较多，需考虑溢出和节约内存空间，但运行速度更快；

分支限界法 VS 回溯法

- 不同点:

- 搜索方式

- 回溯法: 深度优先;
 - 分支限界法: 广度优先;

- 搜索策略

- 回溯法: 根据剪枝函数, 选择下一个扩展接点并按深度优先方式进行搜索;
 - 分支限界法: 在扩展结点处, 先产生其所有的子结点 (分支), 然后根据限界函数, 确定哪些子结点将导致不可行解或非最优解, 将这些子结点剔除, 用剩下的子结点构造当前的活结点表, 然后从该表中取一个结点作为当前扩展结点, 并重复上述过程;

分支限界法的主要分类

- 分支限界法的主要分类

- 根据从活结点表中选择下一个扩展结点的方式

- 队列式**FIFO**分支限界法

- **算法思想**：将活结点表组织成一个队列，并按队列的先进先出**FIFO**原则选取下一个结点作为当前扩展结点；

- 优先队列式分支限界法

- **算法思想**：将活结点表组织成一个优先队列，并按优先队列中规定的结点优先级，选取剩余队列中优先级最高的下一个结点作为当前扩展结点；

- 使用最优队列分支限界法，用类似贪心策略的方式选择解的搜索方向，可以尽快找到近似最优解；
- 由于在搜索到某个层面发现解不够优，还可以从更低的树层次获得问题的更优解，因而能获得比贪心算法更优的解；

第7章 概率算法

概率算法的主要类型

- 概率算法的主要类型
 - 数值概率算法：求 π 值；
 - 蒙特卡罗算法：求主元素；
 - 拉斯维加斯算法：求解n皇后问题；
 - 舍伍德算法：快速排序法的改进；

- 考虑产生 $1, 2, \dots, n$ 的一个随机排列。请简要描述一个时间复杂度为 $O(n)$ 的随机算法的思路。
- 答：一种随机算法的基本思路是：先初始化 $A[i]=i$, $1 \leq i \leq n$ ；然后用线性同余法生成随机数，根据随机数在 A 中依次选两个元素出来交换，此操作执行 n 次。显然，这种算法的时间复杂度为 $O(n)$ 。